9i. Static Vectors for Small Collections

Martin Alfaro
PhD in Economics

INTRODUCTION

Repeated vector creation can quickly become a performance bottleneck, due to the memory-allocation overhead it involves. The issue has far-reaching implications, since vector creation doesn't just occur when a vector is explicitly defined. It also happens internally in various scenarios, such as when referencing a slice like `x[1:2]` in `sum(x[1:2])` or when computing intermediate results on the fly like `x .* y` in `sum(x .* y)`.

The current section introduces one strategy to address this issue, while retaining the convenience of vectors for expressing collections. It leverages the so-called **static vectors**, provided by the `StaticArrays` package. Unlike built-in vectors, which are allocated on the heap, static vectors are stack-allocated.

Internally, static vectors are built on top of tuples. Given the small capacity of the stack, **static vectors are only suitable for collections comprising a few elements**. As a rule of thumb, **you should use static vectors for collections with up to 50 elements**. Exceeding this threshold can lead to increased overhead, potentially offsetting any performance benefits or even resulting in a fatal error. [1]

Despite that static vectors are based on tuples, they offer additional benefits beyond what tuples offer. Firstly, they maintain their performance benefits even at sizes where tuples typically begin to degrade. Secondly, their manipulation closely resembles that of regular vectors, making them more convenient to work with. Third, the `StaticArrays` package extends support to other array types (including matrices) and offer mutable variants. These mutable versions make static vectors more flexible than tuples, which are only available in an immutable form. It's worth indicating though that, while the mutable version provides performance benefits relative to regular vectors, the immutable option still offers the best performance.

> ⚠️ **Warning!**
>
> **The entire section assumes all collections are small**. Thus, all the benefits highlighted are contingent upon this assumption.
>
> We also suppose that the `StaticArrays` package has been loaded by executing `using StaticArrays`. This is why we omit this command from each code snippet.

PERFORMANCE GAINS OF TUPLES FOR SMALL COLLECTIONS

Since static vectors are built on top of tuples, we begin by illustrating the advantages of tuples over vectors. In particular, we emphasize that these advantages only materialize when the collections are small.

Specifically, recall that tuples are immutable and have a fixed size. This allows the compiler to aggressively optimize operations on them. However, as the number of elements grows, performance degrades sharply. In fact, once a tuple exceeds a certain threshold, the computation may fail to complete, causing the program to stall. The underlying reason is that, unlike heap-allocated objects like vectors, tuples live on the stack, which has very limited space.

SMALL COLLECTION

```
_size  = 10
vector = rand(_size)
tuple  = Tuple(vector)

foo(x) = sum(x .* x)
```

```
julia> foo(vector)
  34.806 ns (2 allocations: 144 bytes)
julia> foo(tuple)
  1.742 ns (0 allocations: 0 bytes)
```

MEDIUM COLLECTION

```
_size  = 200
vector = rand(_size)
tuple  = Tuple(vector)

foo(x) = sum(x .* x)
```

```
julia> foo(vector)
  109.528 ns (2 allocations: 1.625 KiB)
julia> foo(tuple)
  120.992 ns (0 allocations: 0 bytes)
```

BIG COLLECTION

```
_size  = 2_000
vector = rand(_size)
tuple  = Tuple(vector)

foo(x) = sum(x .* x)
```

```
julia> foo(vector)
  1.217 µs (3 allocations: 15.688 KiB)
julia> foo(tuple)
  2.306 µs (0 allocations: 0 bytes)
```

BIG COLLECTION - PROGRAM STALLED

```
_size  = 10_000
vector = rand(_size)
tuple  = Tuple(vector)

foo(x) = sum(x.* x)
```

```
julia> foo(vector)
  8.197 µs (3 allocations: 78.188 KiB)
julia> foo(tuple)
# program keeps working indefinitely
```

Large collections can also trigger a different kind of failure: a stack overflow. This typically occurs when the program exhausts the available stack space. Unlike the gradual slowdown caused by performance degradation, a stack overflow produces an immediate and fatal error. In such cases, Julia aborts execution and reports the problem directly, rather than allowing the computation to hang indefinitely.

STACK OVERFLOW

```
function foo(n)
    if n == 0
        ()
    else
        (n, foo(n-1)...)
    end
end
```

```
julia> foo(50_000)
ERROR: StackOverflowError
```

STATIC VECTORS

The package `StaticArrays` includes several variants of static arrays. They preserve the advantages of tuples, while extending their functionality.

Our primary focus here is on the type `SVector`, whose objects we'll simply refer to as SVectors. Like tuples, **SVectors are immutable**, meaning their elements can't be added, removed, or modified.

There are two approaches to creating an SVector, each serving a distinct purpose. The first one creates an SVector directly from literal values, while the second converts an existing regular vector into an SVector. The following examples illustrate both approaches in practice.

VIA LITERAL VALUES

```
# all 'sx' define the same static vector '[3,4,5]'


sx = SVector(3,4,5)
sx = SVector{3, Int64}(3,4,5)
sx = SA[3,4,5]
sx = @SVector [i for i in 3:5]
```

```
julia> sx
3-element SVector{3, Int64} with indices SOneTo(3):
 3
 4
 5
```

VIA EXISTING VECTOR

```
# all 'sx' define a static vector with same elements as 'x'
x = collect(1:10)

sx = SVector(x...)
sx = SVector{length(x), eltype(x)}(x)
sx = SA[x...]
sx = @SVector [a for a in x]
```

```
julia> sx
10-element SVector{10, Int64} with indices SOneTo(10):
  1
  2
  3
  ⋮
  9
 10
```

Of these approaches to creating SVectors, we'll primarily rely on the function `SVector`, occasionally employing `SA` for indexing purposes. [2] Note that the use of **splat operator** `...` is necessary when converting an existing vector into an SVector. This operator expands a collection into a sequence of arguments. For instance, if `x` is a vector or tuple with 3 elements, `foo(x...)` is equivalent to `foo(x[1], x[2], x[3])`.

Regarding slices of SVectors, the **method used for indexing determines whether the resulting slice is a regular vector or an SVector**: a slice remains an SVector when indices are provided as SVectors, whereas the slice becomes a regular vector when indices are ranges or regular vectors. The

sole exception to this rule is when the slice references the whole object (i.e., `sx[:]`), in which case an SVector is returned.

SLICES AS SVECTORS

```
x      = collect(3:10)
sx     = SVector(x...)

# both define static vectors
slice1 = sx[:]
slice2 = sx[SA[1,2]]        # or slice2 = sx[SVector(1,2)]
```

```
julia> slice1
8-element SVector{8, Int64} with indices SOneTo(8):
  3
  4
  ⋮
  9
 10
julia> slice2
2-element SVector{2, Int64} with indices SOneTo(2):
 3
 4
```

WRONG APPROACH

```
x      = collect(3:10)
sx     = SVector(x...)

# both define and ordinary vector
slice1 = sx[1:2]
slice2 = sx[[1,2]]
```

```
julia> slice
2-element Vector{Int64}:
 3
 4
```

SVECTORS DON'T ALLOCATE MEMORY AND ARE FASTER

Since SVectors are internally implemented on top of tuples, they don't allocate memory.

COPY

```
x   = rand(10)


function foo(x)
    a = x[1:2]              #   allocation (copy of slice)
    b = [3,4]               #   allocation (vector creation)

    sum(a) * sum(b)         # 0 allocation (scalars don't allocate)
end
```

```
julia> @btime foo($x)
  42.844 ns (3 allocations: 128 bytes)
```

VIEW

```
x   = rand(10)


@views function foo(x)
    a = x[1:2]              # 0 allocation (view of slice)
    b = [3,4]               #   allocation (vector creation)

    sum(a) * sum(b)         # 0 allocation (scalars don't allocate)
end
```

```
julia> @btime foo($x)
  17.411 ns (1 allocations: 48 bytes)
```

TUPLE

```
x   = rand(10)
tup = Tuple(x)

function foo(x)
    a = x[1:2]              # 0 allocation (slice of tuple)
    b = (3,4)               # 0 allocation (tuple creation)

    sum(a) * sum(b)         # 0 allocation (scalars don't allocate)
end
```

```
julia> @btime foo($tup)
  1.909 ns (0 allocations: 0 bytes)
```

STATIC VECTOR

```
x   = rand(10)
sx  = SA[x...]

function foo(x)
    a = x[SA[1,2]]          # 0 allocation (slice of static array)
    b = SA[3,4]             # 0 allocation (static array creation)

    sum(a) * sum(b)         # 0 allocation (scalars don't allocate)
end
```

```
julia> @btime foo($sx)
  1.821 ns (0 allocations: 0 bytes)
```

The fact that SVectors don't allocate is especially relevant in operations that generate temporary vectors. For instance, this is the case when a reduction requires an intermediate transformation of the vector being reduced.

VECTOR

```
x      = rand(10)


foo(x) = sum(2 .* x)
```

```
julia> @btime foo($x)
  36.695 ns (2 allocations: 144 bytes)
```

SVECTOR

```
x      = rand(10)
sx     = SVector(x...)

foo(x) = sum(2 .* x)
```

```
julia> @btime foo($sx)
  1.822 ns (0 allocations: 0 bytes)
```

The performance benefits of SVectors extend beyond the absence of memory allocation. This is why, even if operations on regular vectors don't allocate memory, SVectors still offer significant speedups. The point is demonstrated below through the function `sum(f, <vector>)`, which adds all elements of `<vector>` after they're transformed via `f`. In the example, the implementation with SVectors yields faster execution times, even when that of regular vectors already avoids memory allocations.

VECTOR

```
x      = rand(10)


foo(x) = sum(a -> 10 + 2a +  3a^2, x)
```

```
julia> @btime foo($x)
  9.420 ns (0 allocations: 0 bytes)
```

SVECTOR

```
x      = rand(10)
sx     = SVector(x...)

foo(x) = sum(a -> 10 + 2a +  3a^2, x)
```

```
julia> @btime foo($sx)
  1.822 ns (0 allocations: 0 bytes)
```

SVECTOR TYPE AND ITS MUTABLE VARIANT

SVectors are immutable, and so their elements can't be added, removed, or modified. While this property is essential for enabling certain compiler optimizations, it also reduces their scope of application. To address this limitation, the package `StaticArrays` additionally provides a mutable variant given by the `MVector` type. The creation of MVectors and their slices follow the same syntax as SVectors, but with the function `SVector` replaced with `MVector`. This is illustrated below.

IMMUTABLE STATIC VECTOR (SVECTOR)

```
x     = [1,2,3]
sx    = SVector(x...)

sx[1] = 0
```

```
ERROR: setindex!(::SVector{3, Int64}, value, ::Int) is not defined
```

MUTABLE STATIC VECTOR (MVECTOR)

```
x     = [1,2,3]
mx    = MVector(x...)

mx[1] = 0
```

```
julia> mx
3-element MVector{3, Int64} with indices SOneTo(3):
 0
 2
 3
```

Because MVectors are mutable, they're well-suited for scenarios where a vector must be initialized and then filled iteratively via a for-loop. In fact, executing `similar(sx)` automatically returns an MVector when `sx` is an SVector.

'SIMILAR' FOR SVECTORS

```
sx = SVector(1,2,3)

mx = similar(sx)        # it defines an MVector with undef elements
```

```
julia> mx
3-element MVector{3, Int64} with indices SOneTo(3):
 129635981728016
 129635981728000
               2
```

TYPE STABILITY: SIZE IS PART OF THE STATIC VECTOR'S TYPE

Formally, SVectors are defined as objects with type `SVector{N,T}`, where `N` specifies the number of elements and `T` denotes the element's type. For instance, `SVector(4,5,6)` has type `SVector{3,Int64}`, since it comprises 3 elements with type `Int64`. The example illustrates that **the number of elements is part of the `SVector` type**. This behavior, inherited from tuples and shared with MVectors, can readily introduce type instabilities if not handled carefully.

To preserve type stability when working with SVectors, we must then employ strategies similar to those employed for tuples. In practice, this means either passing SVectors directly as function arguments, or dispatching on the number of elements using the `Val` technique.

TYPE UNSTABLE

```
x  = rand(50)


function foo(x)

    output = MVector{length(x), eltype(x)}(undef)

    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end

@code_warntype foo(x)                    # type unstable
```

SVECTOR AS FUNCTION ARGUMENT (TYPE STABLE)

```
x  = rand(50)
sx = SVector(x...)

function foo(x)

    output = MVector{length(x), eltype(x)}(undef)

    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end

@code_warntype foo(sx)                    # type stable
```

VAL (TYPE STABLE)

```
x  = rand(50)


function foo(x, ::Val{N}) where N
    sx     = SVector{N, eltype(x)}(x)
    output = MVector{N, eltype(x)}(undef)

    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end

@code_warntype foo(x, Val(length(x)))     # type stable
```

PERFORMANCE COMPARISON

While MVectors offer performance benefits over regular vectors, bear in mind that they're never more performant than SVectors. For this reason, the general guideline is to prefer an SVector if the collection won't be mutated, reserving MVectors for scenarios that require in-place modifications.

The examples below highlight these principles. They show that SVectors and MVectors either deliver a similar performance or SVectors are clearly superior. Furthermore, both SVectors and MVectors consistently outperform regular vectors for small collections.

SIMILAR PERFORMANCE OF SVECTOR AND MVECTOR

```
x      = rand(10)
sx     = SVector(x...)
mx     = MVector(x...)

foo(x) = sum(a -> 10 + 2a +  3a^2, x)
```

```
julia> @btime foo($x)
  9.426 ns (0 allocations: 0 bytes)
julia> @btime foo($sx)
  1.821 ns (0 allocations: 0 bytes)
julia> @btime foo($mx)
  6.614 ns (0 allocations: 0 bytes)
```

SVECTOR BETTER

```
x      = rand(10)
sx     = SVector(x...)
mx     = MVector(x...)

foo(x) = 10 + 2x +  3x^2
```

```
julia> @btime foo.($x)
  35.112 ns (2 allocations: 144 bytes)
julia> @btime foo.($sx)
  1.909 ns (0 allocations: 0 bytes)
julia> @btime foo.($mx)
  14.586 ns (1 allocations: 96 bytes)
```

STATIC VECTORS VS PRE-ALLOCATIONS

Considering the advantages of static vectors over regular vectors, let's compare their performance with other strategies for reducing memory allocations. In particular, we'll examine how they stack up against **pre-allocating memory** for intermediate outputs. Our examples demonstrate that static vectors can efficiently store intermediate results, making pre-allocation techniques unnecessary. Moreover, storing the final output in an MVector can lead to performance gains over using a regular vector.

For the illustration, consider a reduction via for-loop that requires an intermediate result during each iteration `i`. This consists of counting the number of elements in `x` that are greater than `x[i]`. Formally, `x .> x[i]`. To make the comparison stark, we'll isolate the computation of this intermediate step.

Note that every implementation below requires pre-allocating the vector `output`, explaining why each incurs at least one memory allocation. Any additional allocation depends on whether the temporary vector `temp` is also pre-allocated.

NO PRE-ALLOCATION

```
x  = rand(50)


function foo(x; output = similar(x))
    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end
```

```
julia> @btime foo($x)
  2.788 µs (152 allocations: 5.156 KiB)
```

PRE-ALLOCATION

```
x  = rand(50)


function foo(x; output = similar(x), temp = similar(x))
    for i in eachindex(x)
        @. temp      = x > x[i]
           output[i] = sum(temp)
    end

    return output
end
```

```
julia> @btime foo($x)
  1.562 µs (4 allocations: 960 bytes)
```

SVECTOR

```
x  = rand(50)
sx = SVector(x...)

function foo(x; output = Vector{Float64}(undef, length(x)))
    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end
```

```
julia> @btime foo($sx)
  344.444 ns (2 allocations: 480 bytes)
```

SVECTOR AND MVECTOR ('SIMILAR')

```
x  = rand(50)
sx = SVector(x...)

function foo(x; output = similar(x))
    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end
```

```
julia> @btime foo($sx)
  309.357 ns (1 allocations: 448 bytes)
```

SVECTOR AND MVECTOR

```
x  = rand(50)
sx = SVector(x...)

function foo(x; output = MVector{length(x),eltype(x)}(undef))
    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end
```

```
julia> @btime foo($sx)
  308.000 ns (1 allocations: 448 bytes)
```

The "No Preallocation" tab serves as a reference point for comparison with the other methods. As for the "Pre-allocation" tab, it reuses a regular vector to compute `temp`. In contrast, the "SVector" tab converts `x` to an SVector `sx`, without pre-allocating `temp`. This approach is more performant due to the benefits provided by SVectors.

As for the last two tabs, they continue defining `x` as an SVector, but additionally treat `output` as an MVector. One does this by using `similar(sx)` to initialize `output`, whereas the other tab explicitly specifies an MVector. The strategy delivers additional performance gains.

FOOTNOTES

[1.] The recommended number of elements provided is actually lower than the documentation's suggested (100 elements). The reason for this discrepancy is that, as you approach the upper limit, the performance benefits of static vectors compared to regular vectors decrease significantly. As a result, the time spent benchmarking with collections of 100 elements will likely offset any potential advantage.

[2.] The approach based on `@SVector` comes with certain caveats. For instance, it doesn't support defining SVectors based fromlocal variables. In particular, it precludes the use of `eachindex(x)` within an array comprehension, unless `x` is a global variable.